

GSLC Algorithm Design & Analysis Session 6

Name: Wilbert Chandra

NIM: 2602113666

1. Jelaskan pengertian dan tipe graph!

Graph adalah struktur data yang terdiri dari kumpulan simpul (vertex) dan hubungan antar simpul tersebut (edge). Simpul dapat mewakili entitas apa pun, seperti orang, tempat, atau benda. Edge dapat mewakili hubungan apa pun antar entitas, seperti persahabatan, jarak, atau aliran data.

Tipe Graph:

- Directed graph: Edge memiliki arah, sehingga hubungan antar simpul tidak simetris.
- Undirected graph: Edge tidak memiliki arah, sehingga hubungan antar simpul simetris.
- Weighted graph: Edge memiliki bobot yang mewakili biaya atau nilai hubungan antar simpul.
- Unweighted graph: Edge tidak memiliki bobot.

2. Jelaskan perbedaan implementation of graph using cost adjacency matrix dan implementation of graph using cost adjacency list!

Adjacency Matrix:

- Matriks dua dimensi yang menyimpan informasi tentang hubungan antar simpul.
- Kelebihan: mudah untuk memeriksa apakah ada edge antara dua simpul, efisien untuk operasi tertentu seperti floyd-warshall algorithm.
- Kekurangan: boros memori untuk graph sparse (banyak simpul dengan sedikit edge), tidak efisien untuk operasi penyisipan dan penghapusan edge.

Adjacency List:

- Daftar tertaut untuk setiap simpul, yang menyimpan simpul-simpul yang terhubung dengannya.
- Kelebihan: hemat memori untuk graph sparse, efisien untuk operasi penyisipan dan penghapusan edge.
- Kekurangan: lebih sulit untuk memeriksa apakah ada edge antara dua simpul, tidak efisien untuk operasi tertentu seperti floyd-warshall algorithm.

3. Jelaskan pengertian dan cara kerja DFS!

DFS adalah algoritma untuk mencari semua simpul dalam graph. Algoritma ini dimulai dari simpul awal dan menjelajahi semua simpul yang terhubung dengannya secara rekursif.

Cara Kerja:

- Tandai simpul awal sebagai visited.
- Untuk setiap simpul yang berdekatan dengan simpul awal dan belum dikunjungi:
- Tandai simpul tersebut sebagai visited.
- Jalankan DFS pada simpul tersebut.

4. Jelaskan pengertian dan cara kerja BFS!

BFS adalah algoritma untuk mencari semua simpul dalam graph. Algoritma ini dimulai dari simpul awal dan menjelajahi semua simpul yang terhubung dengannya secara level-by-level.

Cara Kerja:

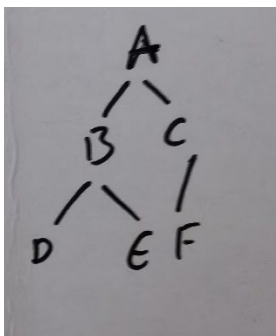
- Buat antrian dan masukkan simpul awal ke dalam antrian.
- Selama antrian tidak kosong:
- Keluarkan simpul dari antrian dan tandai sebagai visited.
- Untuk setiap simpul yang berdekatan dengan simpul yang dikeluarkan dan belum dikunjungi:
 - Tandai simpul tersebut sebagai visited.
 - Masukkan simpul tersebut ke dalam antrian.

5. Jelaskan perbedaan DFS in tree dan DFS in graph! (gambarkan simulasi nya)

DFS dalam Tree:

- Tidak ada siklus
- Tidak perlu melacak kunjungan
- Berakhir saat semua simpul dikunjungi

Misal kita memiliki tree sebagai berikut



Langkah-langkah:

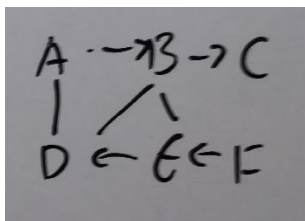
1. Mulai dari root node (A):
 - Tandai A sebagai visited.
 - Push B ke dalam stack.
2. B:
 - Tandai B sebagai visited.
 - Push D ke dalam stack.

3. D:
 - Tandai D sebagai visited.
 - Karena D tidak memiliki anak yang belum dikunjungi, pop D dari stack.
4. B:
 - Karena B tidak memiliki anak yang belum dikunjungi, pop B dari stack.
5. C:
 - Tandai C sebagai visited.
 - Push F ke dalam stack.
6. F:
 - Tandai F sebagai visited.
 - Karena F tidak memiliki anak yang belum dikunjungi, pop F dari stack.
7. C: Karena C tidak memiliki anak yang belum dikunjungi, pop C dari stack.
8. A: Karena A tidak memiliki anak yang belum dikunjungi, pop A dari stack.
9. Hasil: Urutan node yang dikunjungi: A, B, D, C, F

DFS dalam Graph:

- a. Mungkin ada siklus
- b. Perlu melacak kunjungan
- c. Mungkin berhenti ketika mencapai simpul tujuan

Misal kita memiliki graph seperti berikut



Langkah-langkah:

1. **Mulai dari node A:**
 - Tandai A sebagai visited.
 - Push B ke dalam stack.
2. **B:**
 - Tandai B sebagai visited.
 - Push C ke dalam stack.
3. **C:**
 - Tandai C sebagai visited.
 - Karena C tidak memiliki tetangga yang belum dikunjungi, pop C dari stack.
4. **B:**
 - Karena B tidak memiliki tetangga yang belum dikunjungi, pop B dari stack.

5. **A:**

- Push E ke dalam stack.

6. **E:**

- Tandai E sebagai visited.
- Push F ke dalam stack.

7. **F:**

- Tandai F sebagai visited.
- Karena F tidak memiliki tetangga yang belum dikunjungi, pop F dari stack.

8. **E:**

- Karena E tidak memiliki tetangga yang belum dikunjungi, pop E dari stack.

9. **A:**

- Karena A tidak memiliki tetangga yang belum dikunjungi, pop A dari stack.

Hasil:

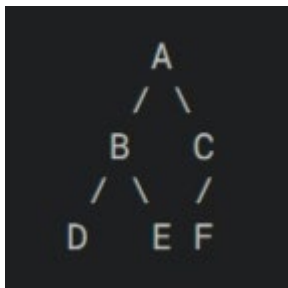
Urutan node yang dikunjungi: A, B, C, E, F

6. Jelaskan perbedaan BFS in tree dan BFS in graph! (gambaran simulasi nya)

BFS in Tree

- Struktur Hierarki
- Tidak ada siklus
- Tidak perlu melacak kunjungan

Misal kita memiliki tree sebagai berikut



Langkah-langkah:

1. **Mulai dari root node (A):**

- Tandai A sebagai visited.
- Masukkan B dan C ke dalam queue.

2. **B:**

- Tandai B sebagai visited.
- Masukkan D dan E ke dalam queue.
- 3. **C:**
 - Tandai C sebagai visited.
 - Masukkan F ke dalam queue.
- 4. **D:**
 - Tandai D sebagai visited.
 - Karena D tidak memiliki anak yang belum dikunjungi, keluarkan D dari queue.
- 5. **E:**
 - Tandai E sebagai visited.
 - Karena E tidak memiliki anak yang belum dikunjungi, keluarkan E dari queue.
- 6. **F:**
 - Tandai F sebagai visited.
 - Karena F tidak memiliki anak yang belum dikunjungi, keluarkan F dari queue.
- 7. **B:**
 - Karena B tidak memiliki anak yang belum dikunjungi, keluarkan B dari queue.
- 8. **C:**
 - Karena C tidak memiliki anak yang belum dikunjungi, keluarkan C dari queue.
- 9. **A:**
 - Karena A tidak memiliki anak yang belum dikunjungi, keluarkan A dari queue.

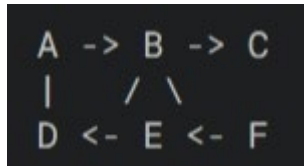
Hasil:

Urutan node yang dikunjungi: A, B, C, D, E, F

BFS in Graph

- Struktur Non-hierarkis
- Mungkin ada siklus
- Perlu melacak kunjungan

Misal kita memiliki graph seperti berikut



Langkah-langkah:

1. **Mulai dari node A:**

- Tandai A sebagai visited.
- Masukkan B dan C ke dalam queue.

2. **B:**

- Tandai B sebagai visited.
- Masukkan E ke dalam queue.

3. **C:**

- Tandai C sebagai visited.
- Masukkan F ke dalam queue.

4. **D:**

- Tandai D sebagai visited.
- Karena D tidak memiliki tetangga yang belum dikunjungi, keluarkan D dari queue.

5. **E:**

- Tandai E sebagai visited.
- Karena E tidak memiliki tetangga yang belum dikunjungi, keluarkan E dari queue.

6. **F:**

- Tandai F sebagai visited.
- Karena F tidak memiliki tetangga yang belum dikunjungi, keluarkan F dari queue.

7. **B:**

- Karena B tidak memiliki tetangga yang belum dikunjungi, keluarkan B dari queue.

8. **C:**

- Karena C tidak memiliki tetangga yang belum dikunjungi, keluarkan C dari queue.

9. **A:**

- Karena A tidak memiliki tetangga yang belum dikunjungi, keluarkan A dari queue.

Hasil:

Urutan node yang dikunjungi: A, B, C, D, E, F

7. Jelaskan concept dari priority queue dan contoh dalam kehidupan sehari-hari!

Priority queue adalah struktur data yang menyimpan elemen dengan prioritas. Elemen dengan prioritas lebih tinggi akan diproses terlebih dahulu.

Contoh dalam Kehidupan Sehari-hari:

- Antrian pasien di rumah sakit: Pasien dengan kondisi yang lebih kritis akan dilayani terlebih dahulu.
- Antrian di bandara: Penumpang dengan status penerbangan yang lebih mendesak (seperti penerbangan yang tertunda) akan dilayani terlebih dahulu.
- Sistem operasi komputer: Proses dengan prioritas lebih tinggi (seperti proses yang kritis untuk sistem) akan dijadwalkan terlebih dahulu.

8. Jelaskan pengertian dan cara kerja heap!

Heap adalah struktur data yang merupakan pohon biner lengkap. Heap memiliki sifat bahwa nilai setiap node selalu lebih besar atau sama dengan nilai anak-anaknya.

Cara Kerja:

- Heapify: Operasi untuk mengubah struktur data menjadi heap.
- Enqueue: Operasi untuk menambahkan elemen baru ke dalam heap.
- Dequeue: Operasi untuk menghapus elemen dengan prioritas tertinggi dari heap.

9. Jelaskan cara kerja Tree Traversal:

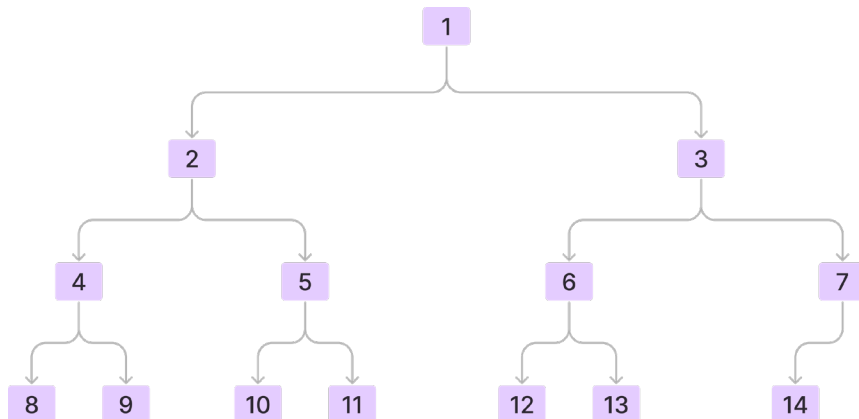
Tree traversal adalah proses mengunjungi semua node dalam tree dan memproses data yang disimpan dalam node tersebut.

Metode Tree Traversal:

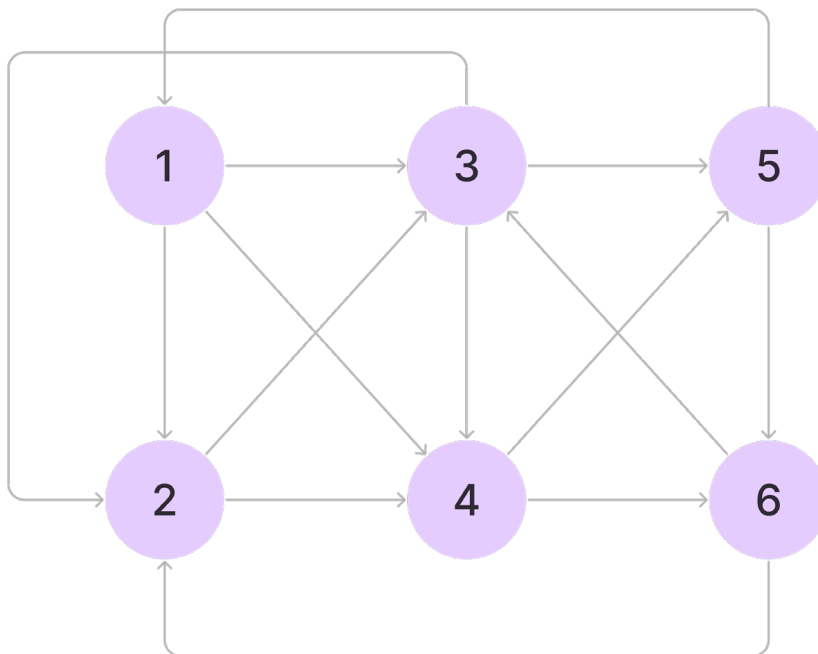
- Pre-order Traversal: Mengunjungi root node terlebih dahulu, kemudian mengunjungi semua node di subtree kiri, dan terakhir mengunjungi semua node di subtree kanan.
- Level-order Traversal: Mengunjungi semua node pada level yang sama terlebih dahulu, kemudian mengunjungi semua node pada level berikutnya, dan seterusnya.
- In-order Traversal: Mengunjungi semua node di subtree kiri terlebih dahulu, kemudian mengunjungi root node, dan terakhir mengunjungi semua node di subtree kanan.
- Post-order Traversal: Mengunjungi semua node di subtree kiri terlebih dahulu, kemudian mengunjungi semua node di subtree kanan, dan terakhir mengunjungi root node.

3	∞	?	∞	∞	∞	∞	∞	∞	10
4	∞	∞	∞	∞	∞	∞	∞	9	0
5	7	∞	∞	∞	∞	∞	∞	∞	∞
6	∞	∞	∞	∞	∞	∞	∞	2	∞
7	∞	∞	∞	5	∞	∞	∞	∞	12
8	∞	4	∞	9	∞	2	3	∞	∞
9	∞	∞	10	∞	∞	∞	∞	∞	3

b. Draw a tree with 14 vertices!



c. Draw a directed acyclic graph with 6 vertices and 14 edges!

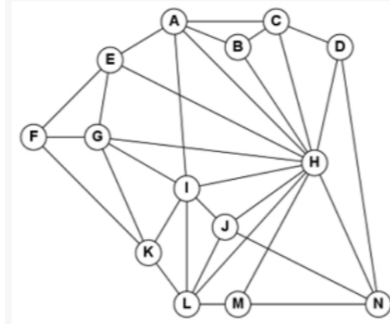


d. Suppose that your computer only has enough memory to store 40000 entries.
Which best graph data structure(s) - you can choose more than 1 should you use to store a simple undirected graph with 200 vertices, 19900 edges, and the existence of $\text{edge}(u,v)$ is frequently asked?

- Adjacency Matrix

- Adjacency List
- Edge List
- Jawaban: **Adjacency List** adalah pilihan terbaik untuk menyimpan graf tak terarah sederhana dengan 200 simpul, 19900 sisi, dan keberadaan sisi (u,v) yang sering ditanyakan karena:
 - Membutuhkan memori yang lebih sedikit dibandingkan Adjacency Matrix.
 - Efisien untuk graf yang jarang.
 - Memudahkan untuk menemukan tetangga suatu simpul.

e. Do implementation of DFS and BFS for the following graph:



Set H sebagai node awal karena paling bercabang

DFS (Rood node : H)	BFS (Rood node : H)
Visited : H Stack : N, M, L, J, I, K, G, F, E, A, B, C, D	Visited : H Stack : N, M, L, J, I, K, G, F, E, A, B, C, D
Visited : H, N Stack : M, L, J, I, K, G, F, E, A, B, C, D	Visited : H, N Stack : M, L, J, I, K, G, F, E, A, B, C, D
Visited : H, N, M Stack : L, J, I, K, G, F, E, A, B, C, D	Visited : H, N, M Stack : L, J, I, K, G, F, E, A, B, C, D
Visited : H, N, M, L Stack : J, I, K, G, F, E, A, B, C, D	Visited : H, N, M, L Stack : J, I, K, G, F, E, A, B, C, D
Visited: H, N, M, L, J Stack: I, K, G, F, E, A, B, C, D	Visited: H, N, M, L, J Stack: I, K, G, F, E, A, B, C, D
Visited: H, N, M, L, J, I Stack: K, G, F, E, A, B, C, D	Visited: H, N, M, L, J, I Stack: K, G, F, E, A, B, C, D
Visited: H, N, M, L, J, I, K Stack: G, F, E, A, B, C, D	Visited: H, N, M, L, J, I, G Stack: K, F, E, A, B, C, D
Visited: H, N, M, L, J, I, K, G Stack: F, E, A, B, C, D	Visited: H, N, M, L, J, I, G, E Stack: K, F, A, B, C, D
Visited: H, N, M, L, J, I, K, G, F Stack: E, A, B, C, D	Visited: H, N, M, L, J, I, G, E, A Stack: K, F, B, C, D
Visited: H, N, M, L, J, I, K, G, F, E Stack: A, B, C, D	Visited: H, N, M, L, J, I, G, E, A, B Stack: K, F, C, D
Visited: H, N, M, L, J, I, K, G, F, E, A Stack: B, C, D	Visited: H, N, M, L, J, I, G, E, A, B, C Stack: K, F, D
Visited: H, N, M, L, J, I, K, G, F, E, A, B Stack: C, D	Visited: H, N, M, L, J, I, G, E, A, B, C, D Stack: K, F
Visited: H, N, M, L, J, I, K, G, F, E, A, B, C Stack: D	Visited: H, N, M, L, J, I, G, E, A, B, C, D, K Stack: F

Visited: H, N, M, L, J, I, K, G, F, E, A, B, C, D Stack:	Visited: H, N, M, L, J, I, G, E, A, B, C, D, K, F Stack :
--	--

- f. Make the steps of the following numbers 34, 15, 23, 9, 41, 26, 39, 11, 7, 28 by using Heap sort!

Heapify -> swap root with max value in the array

Steps	Heap Sort	Array
1	Start Array	[34, 15, 23, 9, 41, 26, 39, 11, 7, 28]
2	Buat max heap	[41, 34, 39, 15, 23, 26, 28, 7, 9, 11]
3	Tukar root element dengan last element dalam array	[11, 34, 39, 15, 23, 26, 28, 7, 9, 41]
4	Heapify pada elemen tersisa	[39, 23, 34, 11, 15, 26, 28, 7, 9, 41]
5	Tukar root element dengan last element dalam heap	[9, 23, 34, 11, 15, 26, 28, 7, 39 , 41]
6	Heapify pada elemen tersisa	[34, 15, 28, 9, 11, 26, 23, 7, 39 , 41]
7	Tukar root element dengan last element dalam heap	[7, 15, 28, 9, 11, 26, 23, 34 , 39 , 41]
8	Heapify pada elemen tersisa	[28, 11, 26, 7, 9, 15, 23, 34 , 39 , 41]
9	Tukar root element dengan last element dalam heap	[23, 11, 26, 7, 9, 15, 28 , 34 , 39 , 41]
10	Heapify pada elemen tersisa	[26, 11, 23, 7, 9, 15, 28 , 34 , 39 , 41]
11	Tukar root element dengan last element dalam heap	[15, 11, 23, 7, 9, 26 , 28 , 34 , 39 , 41]
12	Heapify pada elemen tersisa	[23, 11, 15, 7, 9, 26 , 28 , 34 , 39 , 41]
13	Tukar root element dengan last element dalam heap	[9, 11, 15, 7, 23 , 26 , 28 , 34 , 39 , 41]
14	Heapify pada elemen tersisa	[15, 11, 9, 7, 23 , 26 , 28 , 34 , 39 , 41]
15	Tukar root element dengan last element dalam heap	[7, 11, 9, 15 , 23 , 26 , 28 , 34 , 39 , 41]
16	Heapify pada elemen tersisa	[11, 7, 9, 15 , 23 , 26 , 28 , 34 , 39 , 41]
17	Tukar root element dengan last element dalam heap	[9, 7, 11 , 15 , 23 , 26 , 28 , 34 , 39 , 41]
18	Heapify pada elemen tersisa	[7 , 9 , 11 , 15 , 23 , 26 , 28 , 34 , 39 , 41]